EXPERIMENT : 18

AIM:

Experiment to understand KNN algorithm for a given data set.

DESCRIPTION:

Understand the KNN algorithm for the dataset given.

CODE:

```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/iris.csv')

# Remove extra spaces from column names
df.columns = df.columns.str.strip()

print(df.columns)
print(df.head())

# Features (exclude Id and target column)
features = df.iloc[:, 1:-1].values

# Target label (last column)
label = df.iloc[:, -1].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=42
)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)

print("Training Accuracy:", model_KNN.score(xtrain, ytrain))
print("Testing Accuracy:", model_KNN.score(xtest, ytest))

from sklearn.metrics import confusion_matrix, classification_report

ypred = model_KNN.predict(xtest)

print("Confusion Matrix:")
print(confusion_matrix(ytest, ypred))

print("Classification Report:")
print(classification_report(ytest, ypred))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='object')
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
Training Accuracy: 0.9583333333333334
Testing Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

RESULT:

The K-Nearest Neighbors (KNN) algorithm was successfully implemented on the given dataset. The model classified the data based on the nearest neighbors and predicted the output according to the similarity between data points.